

Quantity v2

a distribution over a condition-space

A Quantity is a function from a shared condition-space to a distribution, non-trivial only along the dimensions it is programmed to be sensitive to.

NORTH-STAR direction — the framework code today still implements v1 (the lookup-table model).

`quantity_v2_design.md` · `PLATFORM_VISION.md`

From v1 to v2

v1 worked, but didn't fit how I actually reason about a circuit

v1 felt unintuitive

values stored as look-up tables of named corners (by_scenario / by_mode), with spread bolted on as an optional field. I kept translating physics into dict keys.



v2 aims to be intuitive & useful

a value is just its distribution over the conditions it actually responds to. the model matches the physics — so worst-case, nominal and Monte-Carlo all fall out.

designed around worst-case circuit analysis as I do it — not a generic units library

the rest of this deck shows what 'intuitive' buys: honest distribution arithmetic, correlated-vs-independent variation, condition-space sensitivities, and one type that holds both the datasheet contract and bench characterization.

One distribution, read three ways

one type, three read-tiers

Worst-case

the [min, max] interval — exact bounds

reads: min, max

Nominal

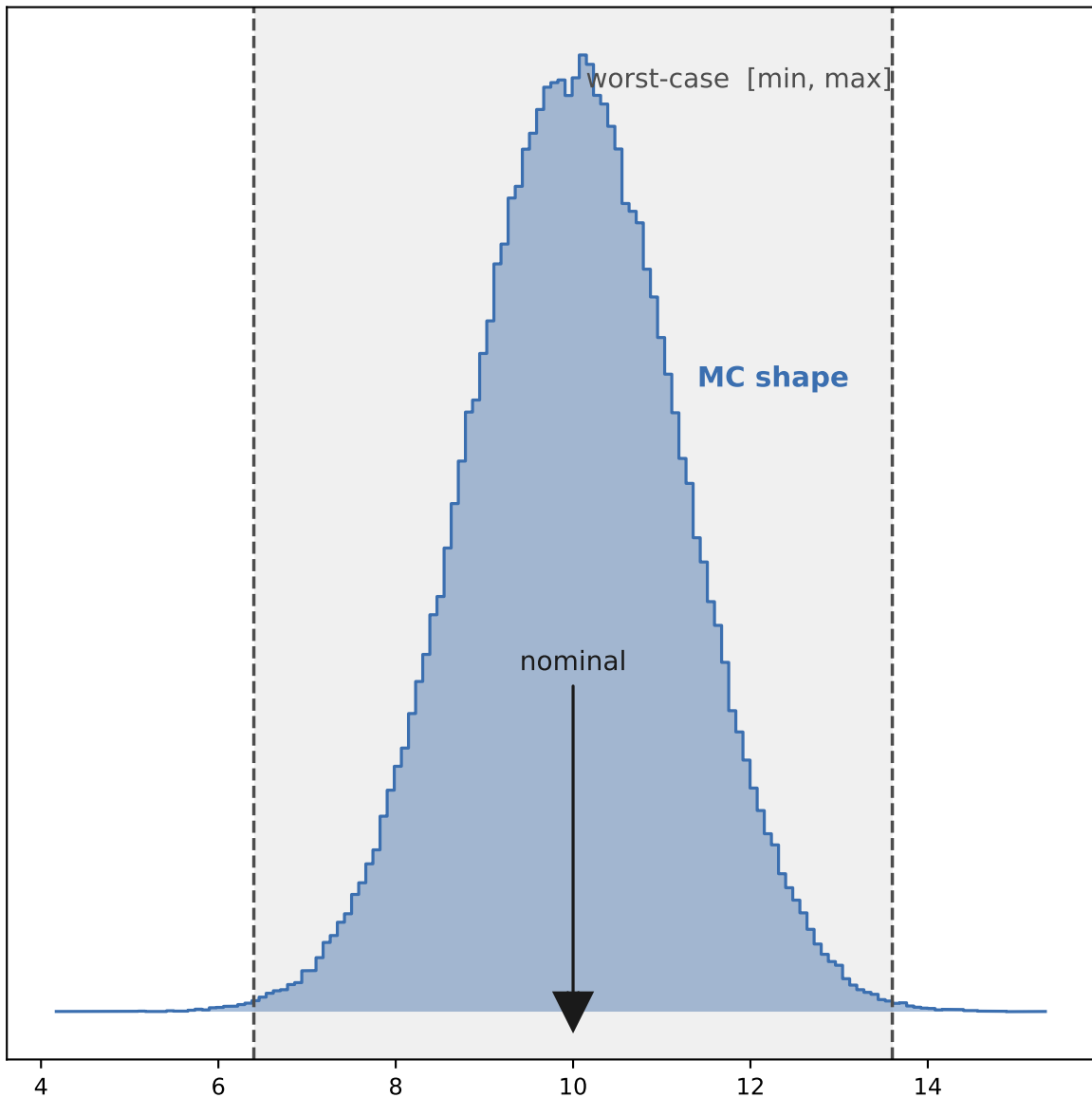
the labelled nom sample — OPTIONAL

reads: nom (or None)

Monte-Carlo

the distribution shape (opt-in)

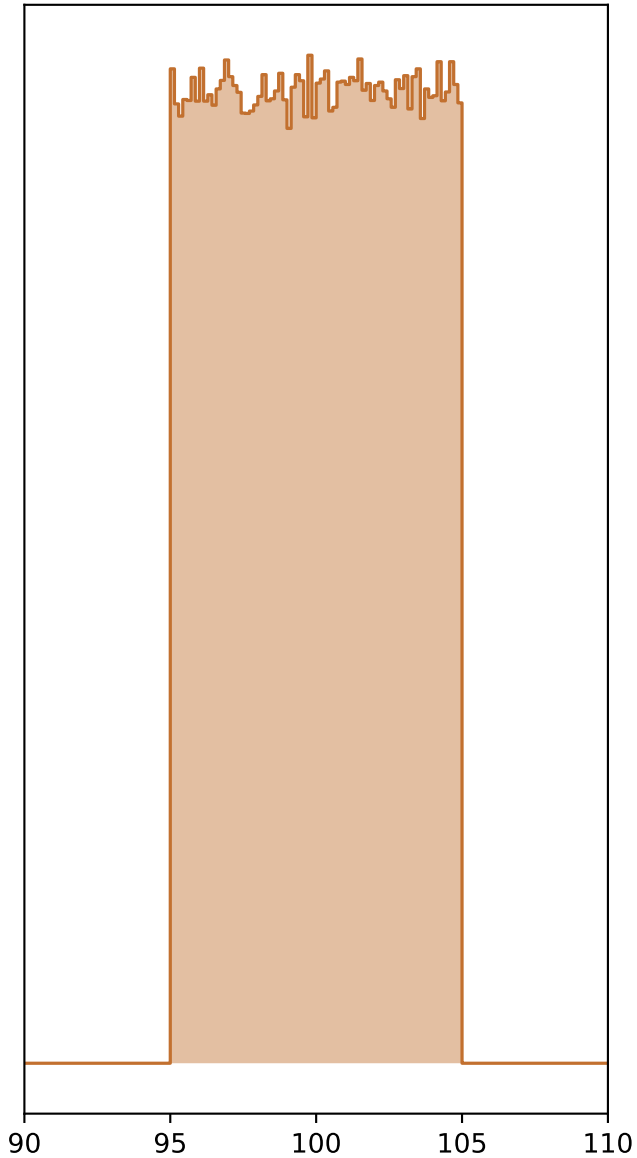
reads: full shape



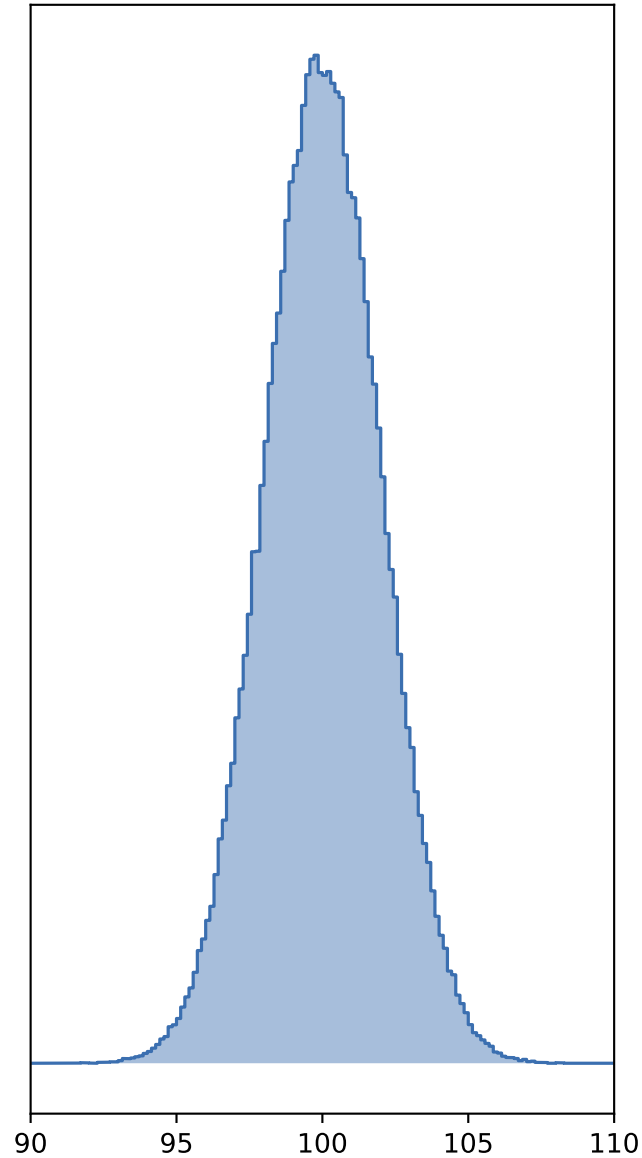
$(min, max) = uniform \cdot nom = optional\ labelled\ sample \cdot invariant\ min \leq nom \leq max$

Distribution forms — shape only matters at the Monte-Carlo tier

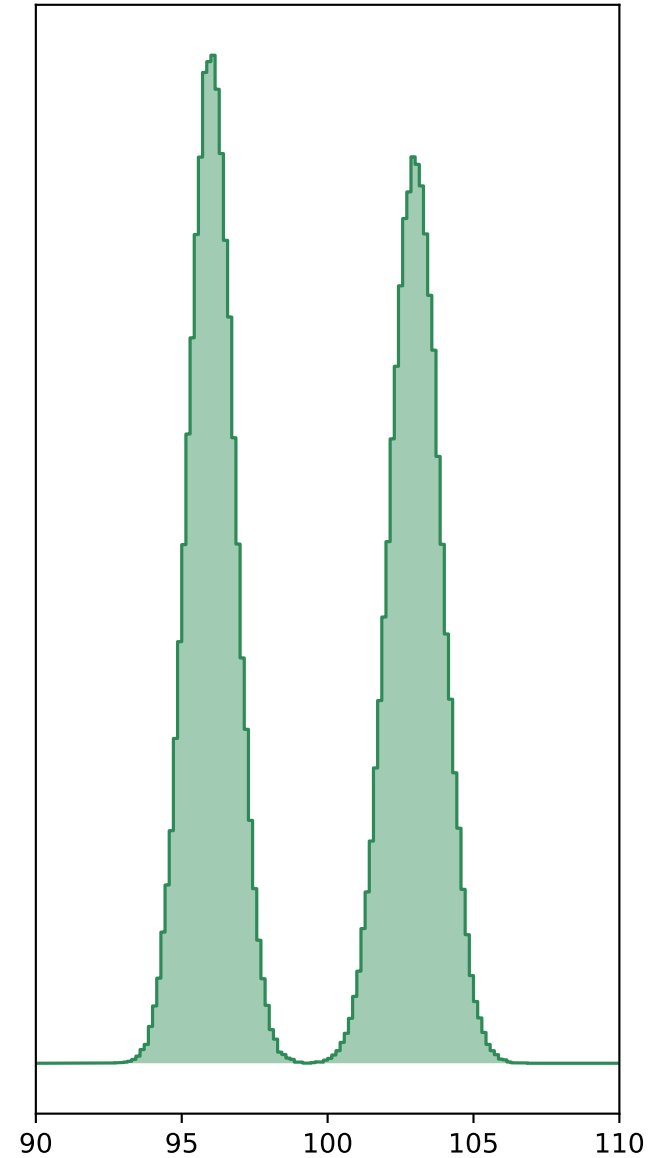
Uniform = the (min, max) interval



Normal



Bimodal



the honest default: 'we know the band, not the shape'

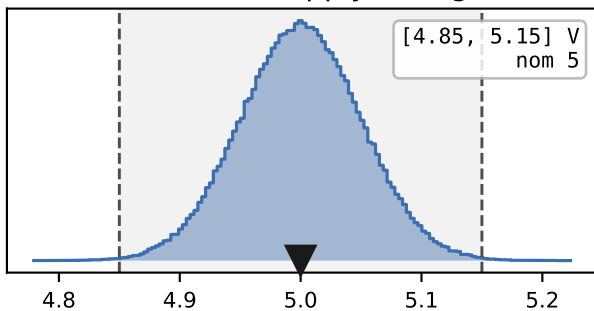
refine to this when characterization justifies it

two-lot / two-population reality — structure no interval can see

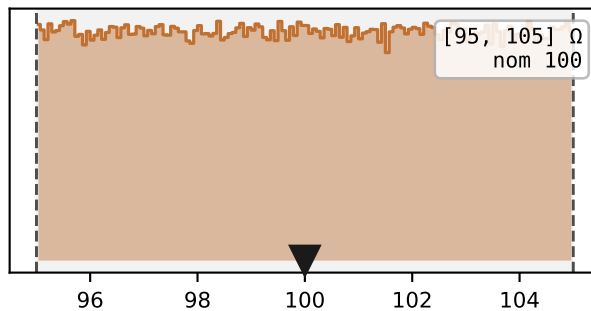
Distribution arithmetic — mixing forms under $+$ $\times$ $\div$

filled = MC shape · dashed band = worst-case interval (interval arithmetic) ▼ = propagated nominal

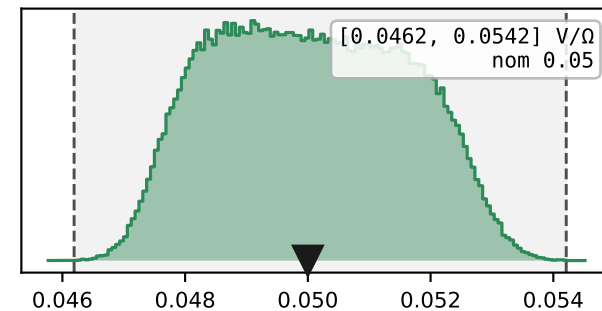
Normal — supply voltage



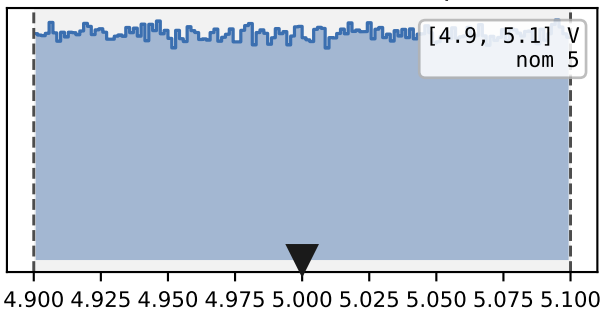
Uniform — resistor tol. band



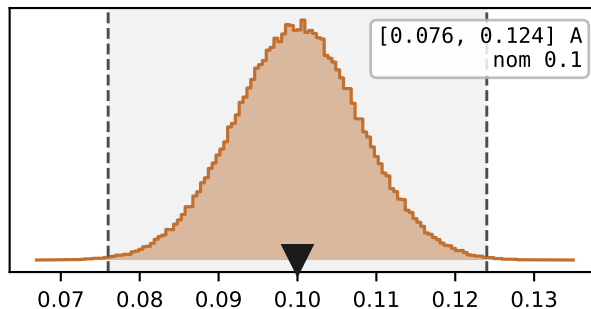
$= V / R \rightarrow$ current (A)



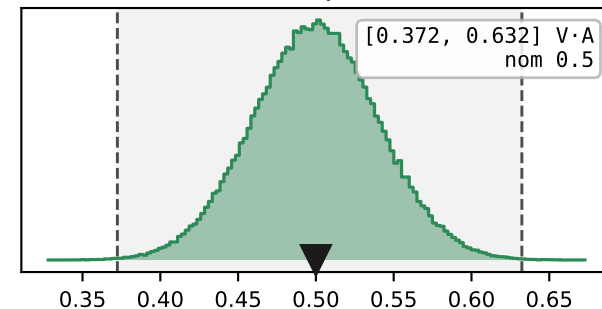
Uniform — rail within spec



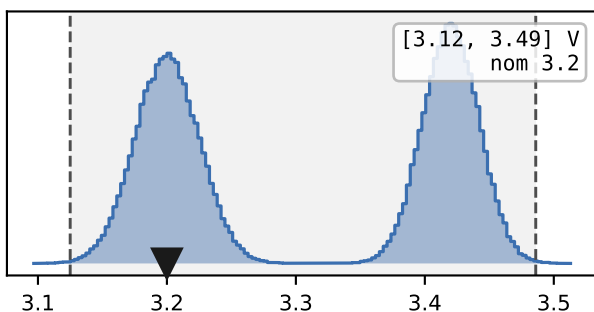
Normal — load current



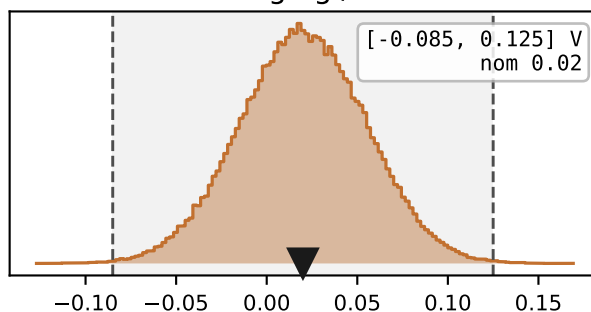
$= V \times I \rightarrow$ power (W)



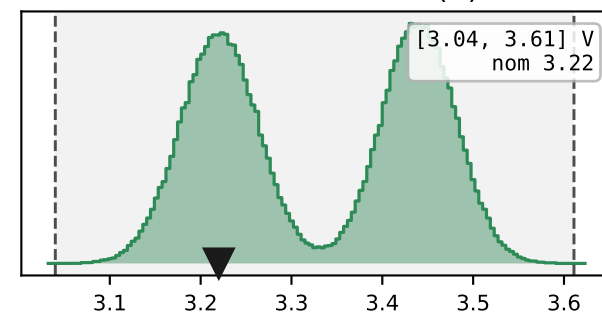
Bimodal — Vref across two lots



Normal — aging / trim offset

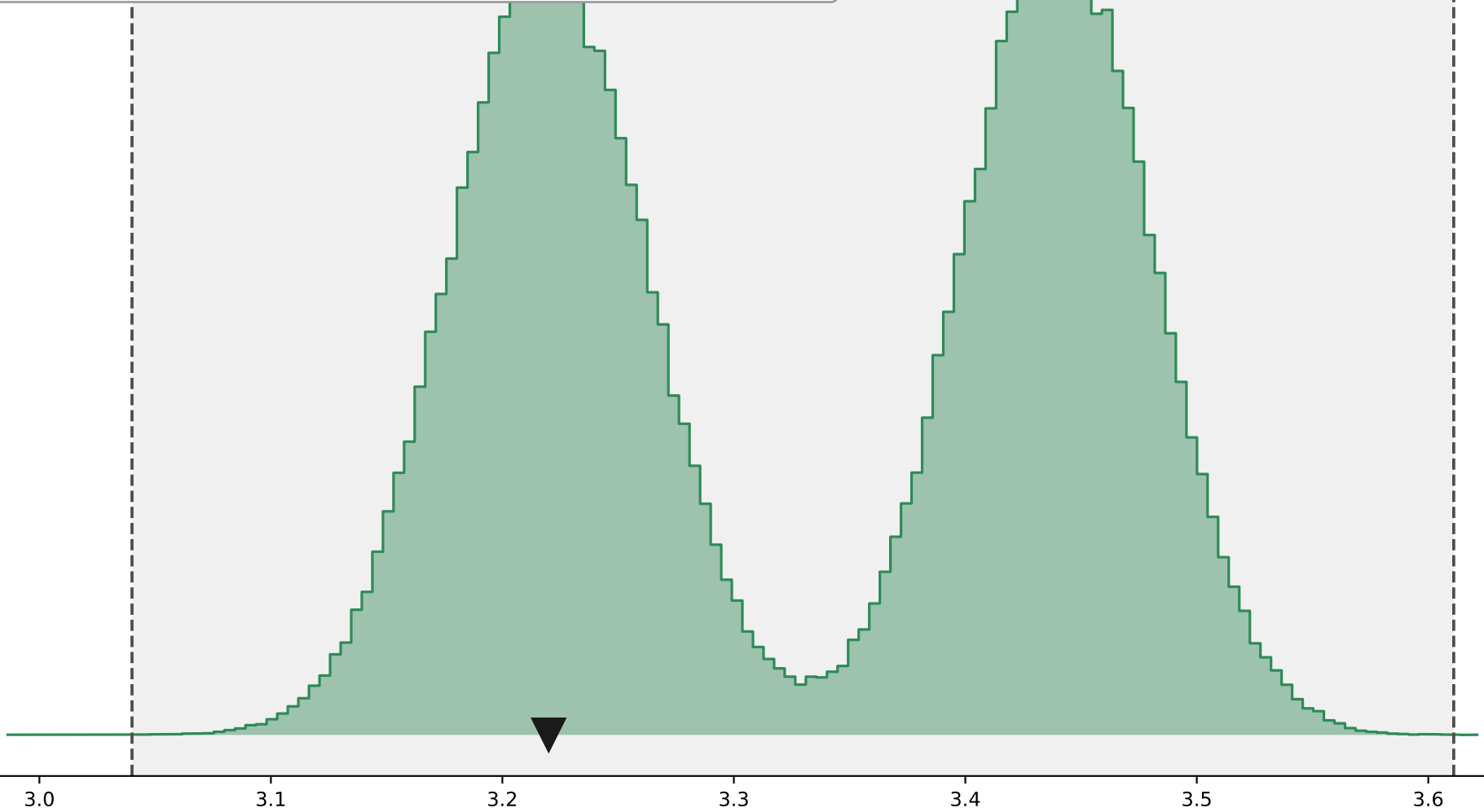


$= V_{\text{ref}} + \text{offset} \rightarrow$ (V)



Closed under worst-case arithmetic

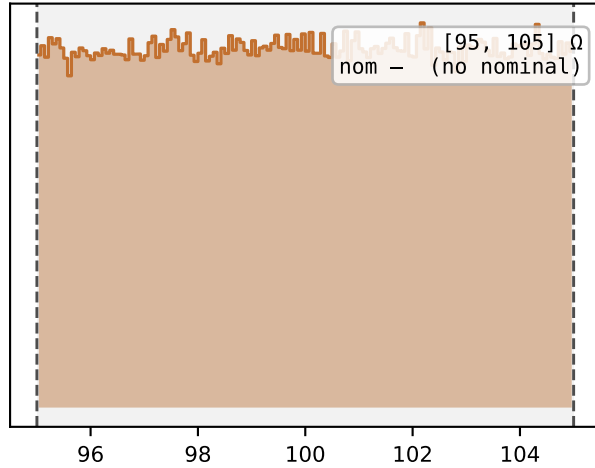
bounds → interval arithmetic on the operands
nominal → ordinary scalar arithmetic (if both declare one)
shape → Monte-Carlo (sampled)



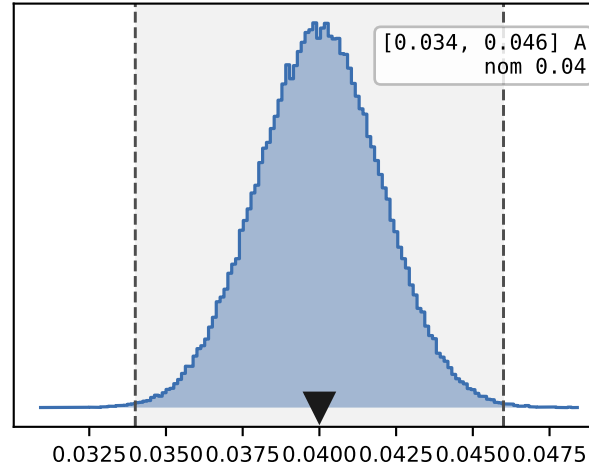
the dashed worst-case band always contains the MC shape — (min, max, nom) propagates cheaply without tracking the true convolved PDF

Optional nominal — explicit opt-in, never a silent fallback

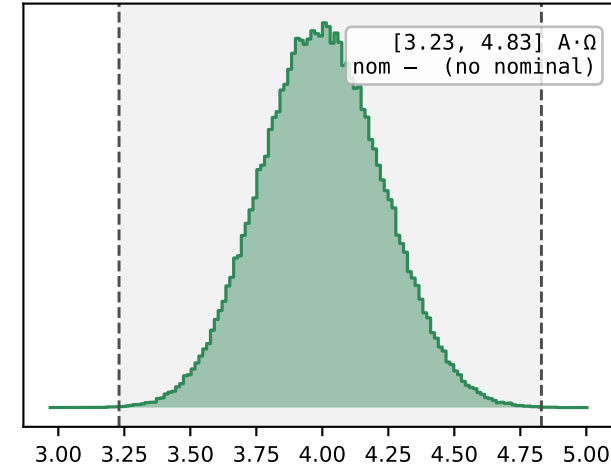
Resistor — interval only (min/max, no typ)



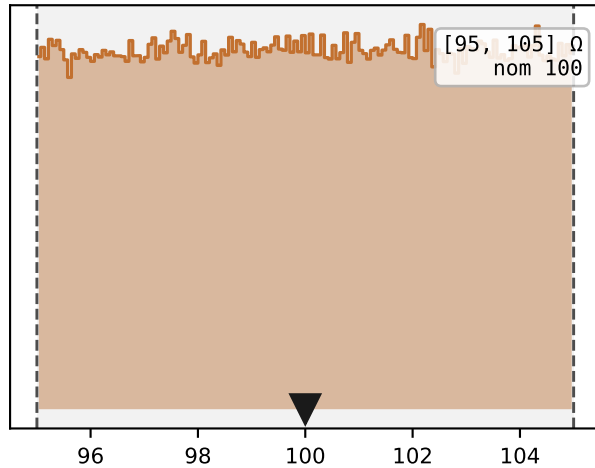
Current — Normal (nom declared)



$= I \times R \rightarrow$ nominal tier UNDEFINED



R.with_nominal(100 Ω) — explicit opt-in

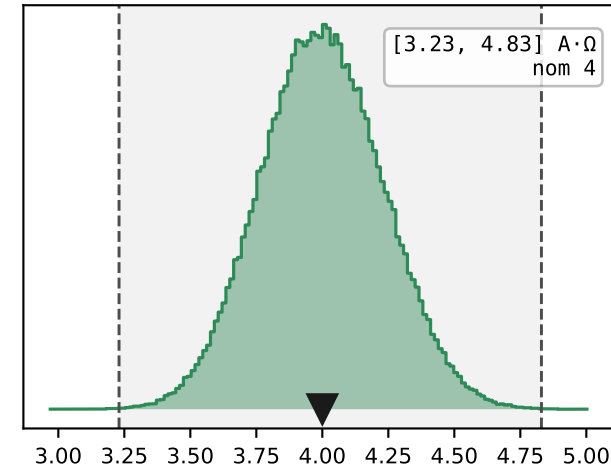


nominal propagates ONLY if both operands declare one.

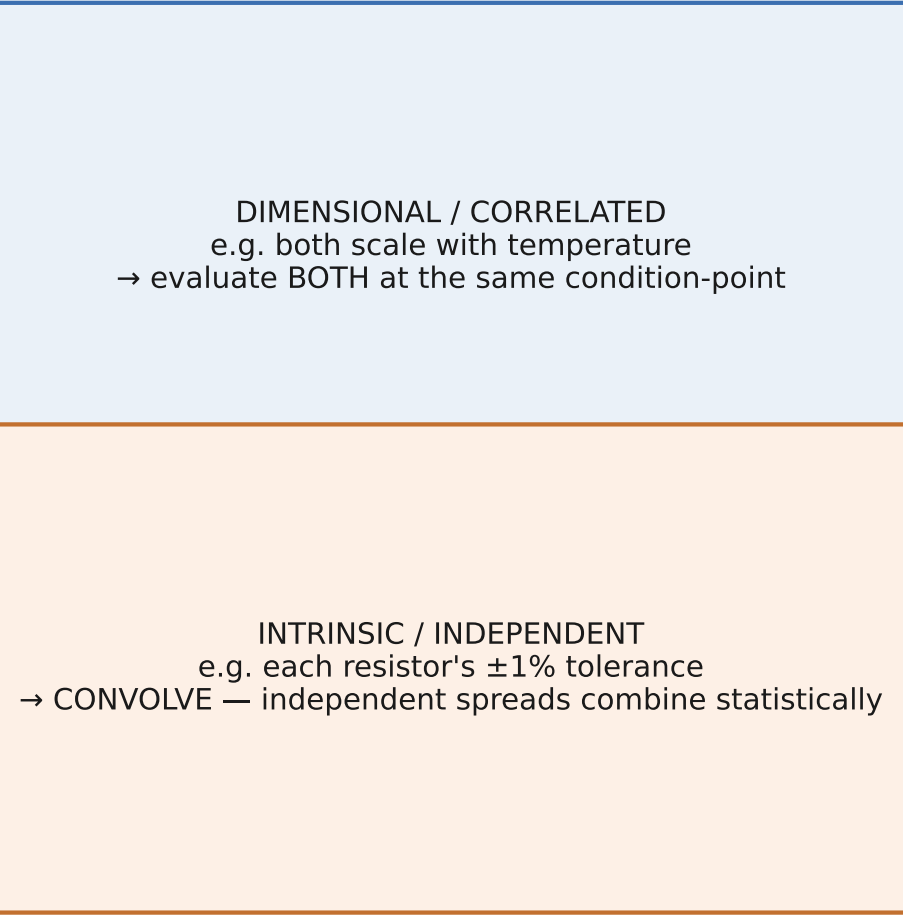
no automatic midpoint fallback — $(\min + \max)/2 \neq \text{typ}$ in general.

opt in explicitly, once, where a nominal-tier number is needed.

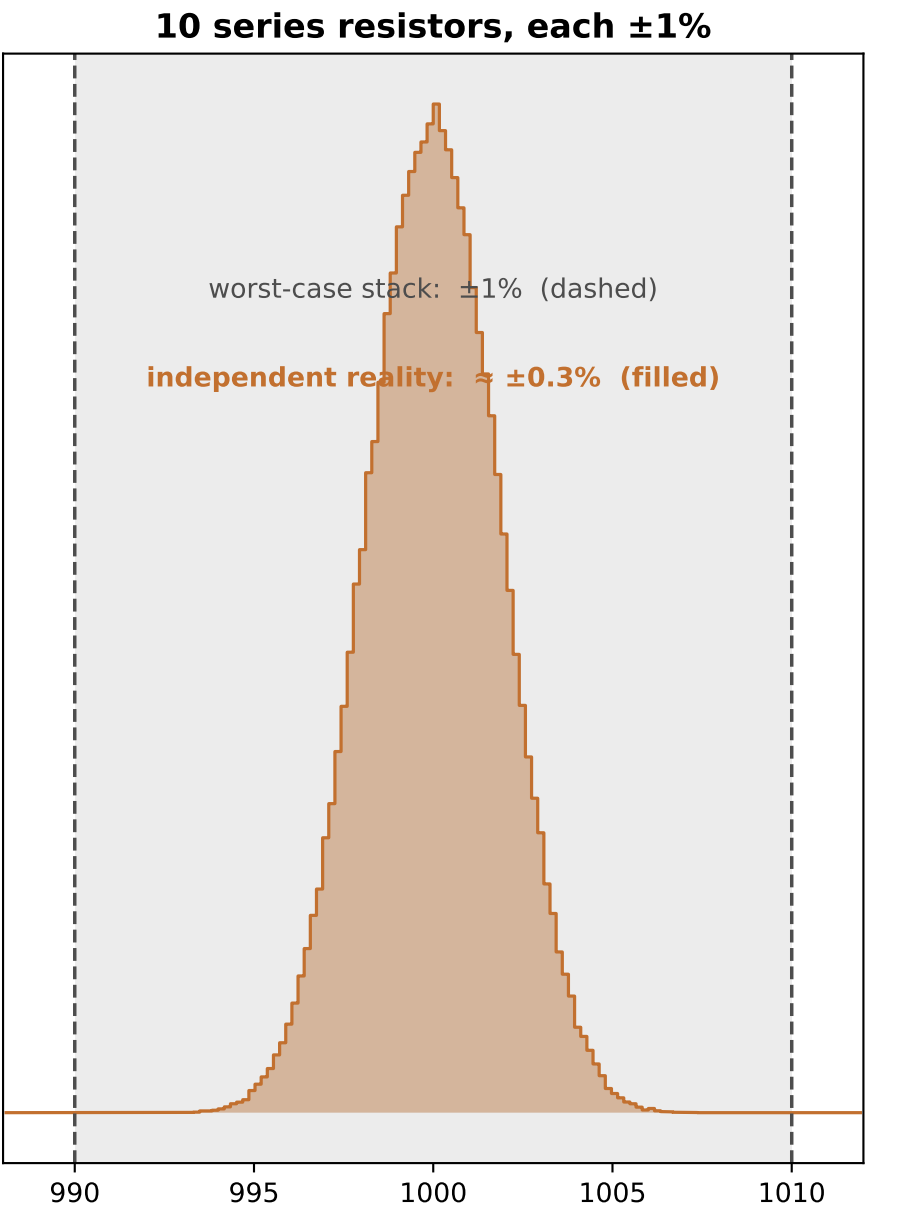
$= I \times R \rightarrow$ nom 4.0 V



Two kinds of variation



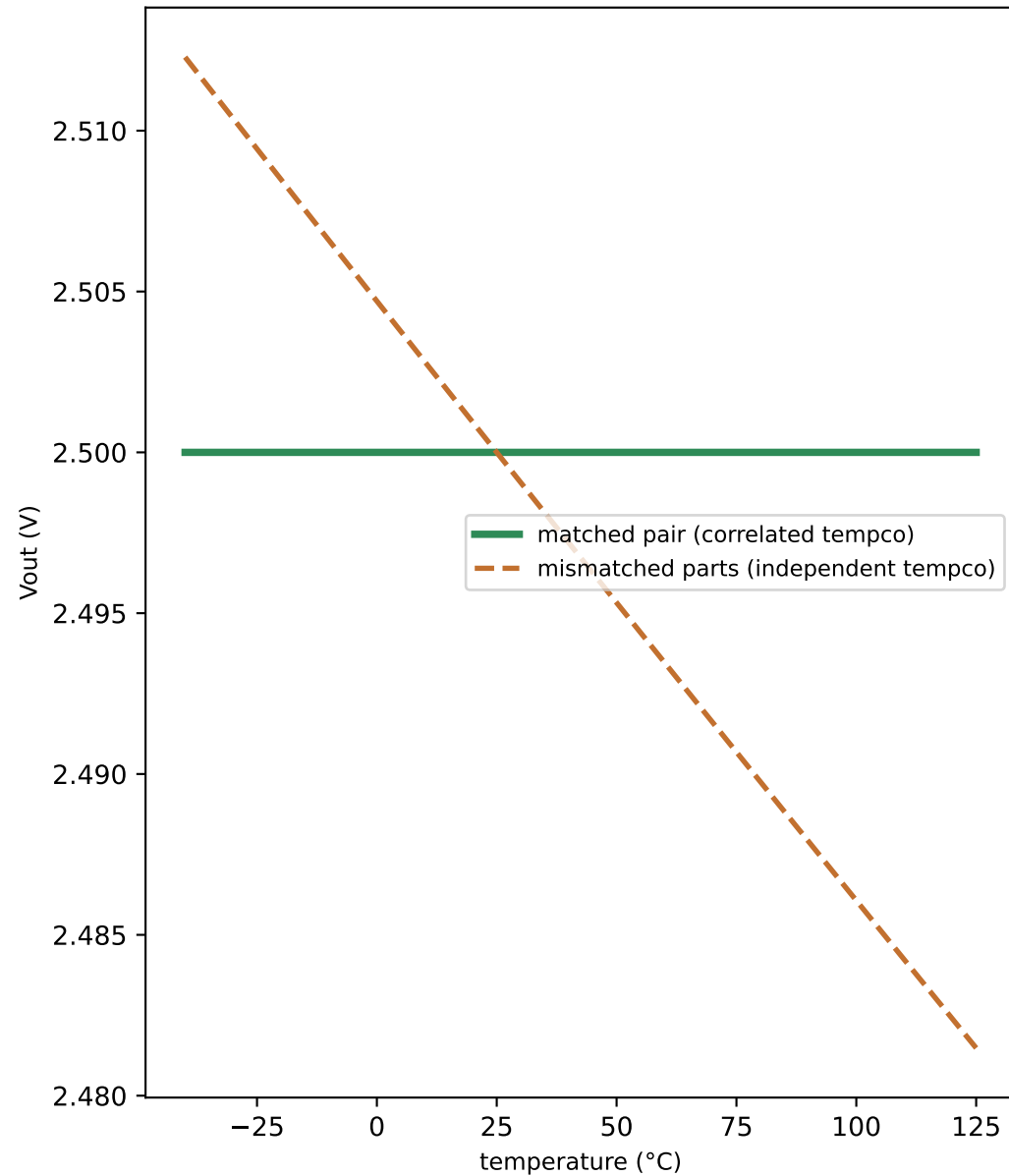
$$\text{Quantity} \approx \text{intrinsic_distribution} + \{ \text{dimension: sensitivity} \}$$



worst-case stacking independents is wildly pessimistic — v2 convolves them

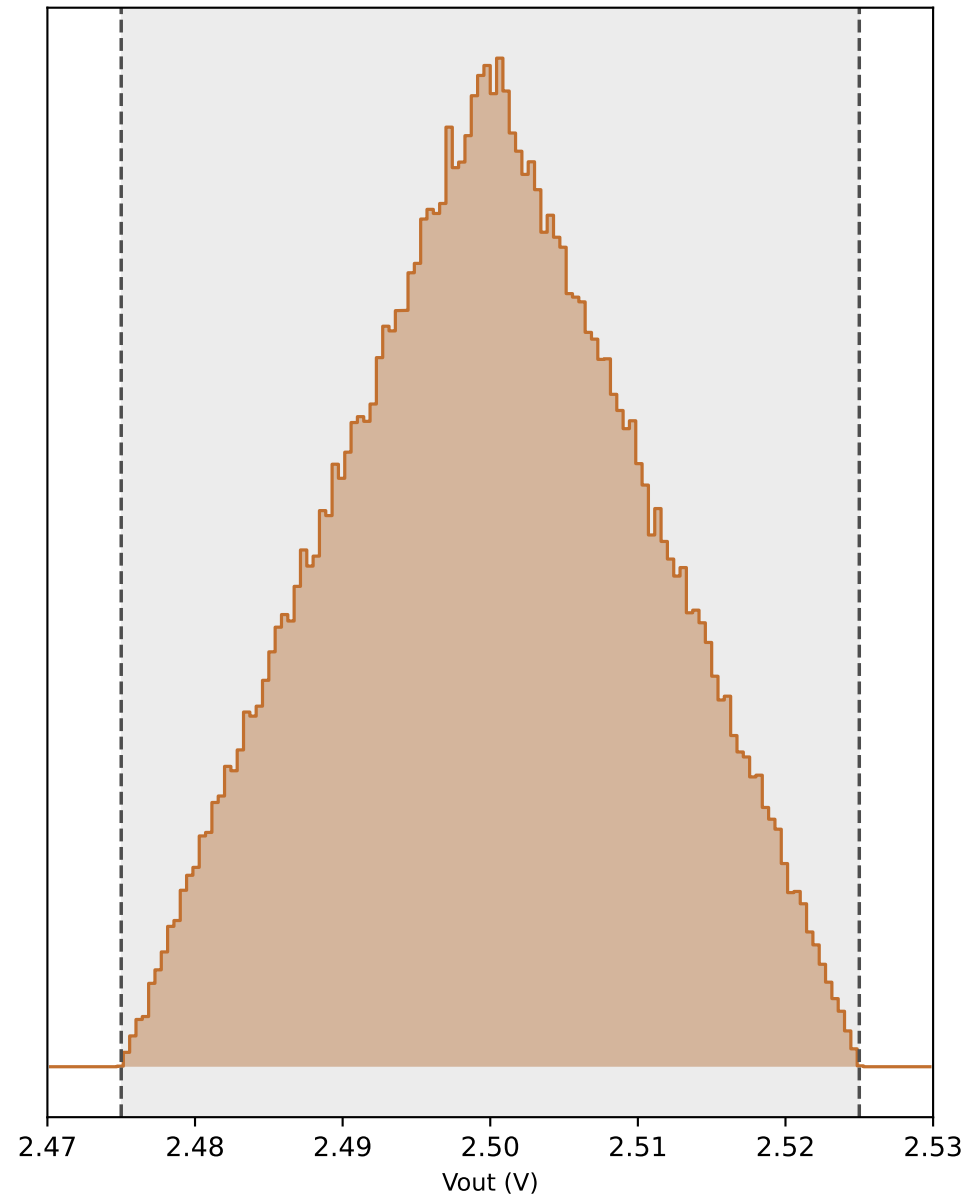
Worked example — a 10 k Ω : 10 k Ω resistor divider

Correlated — tempco cancels in a ratio



*matched pair drifts together $\rightarrow$ ratio invariant
(evaluate both at one condition-point)*

Independent — tolerances convolve



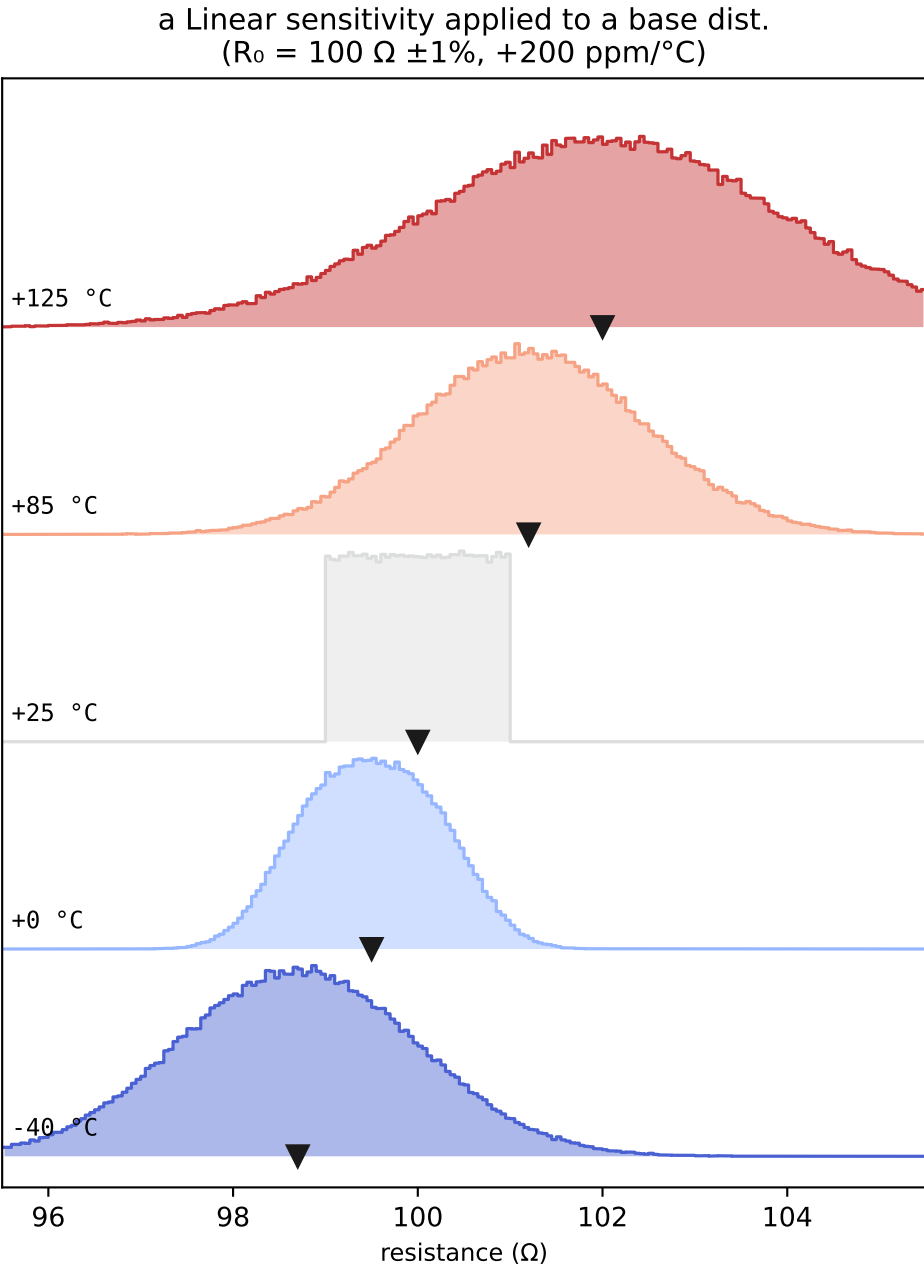
*independent $\pm 1\%$ $\rightarrow$ triangular, mass near 2.5 V
worst-case (dashed) keeps only the corners*

Condition-space & sensitivities

condition-space = the named dimensions a project shares:
{ temperature, vbat, mode, ... }, each with corner values.

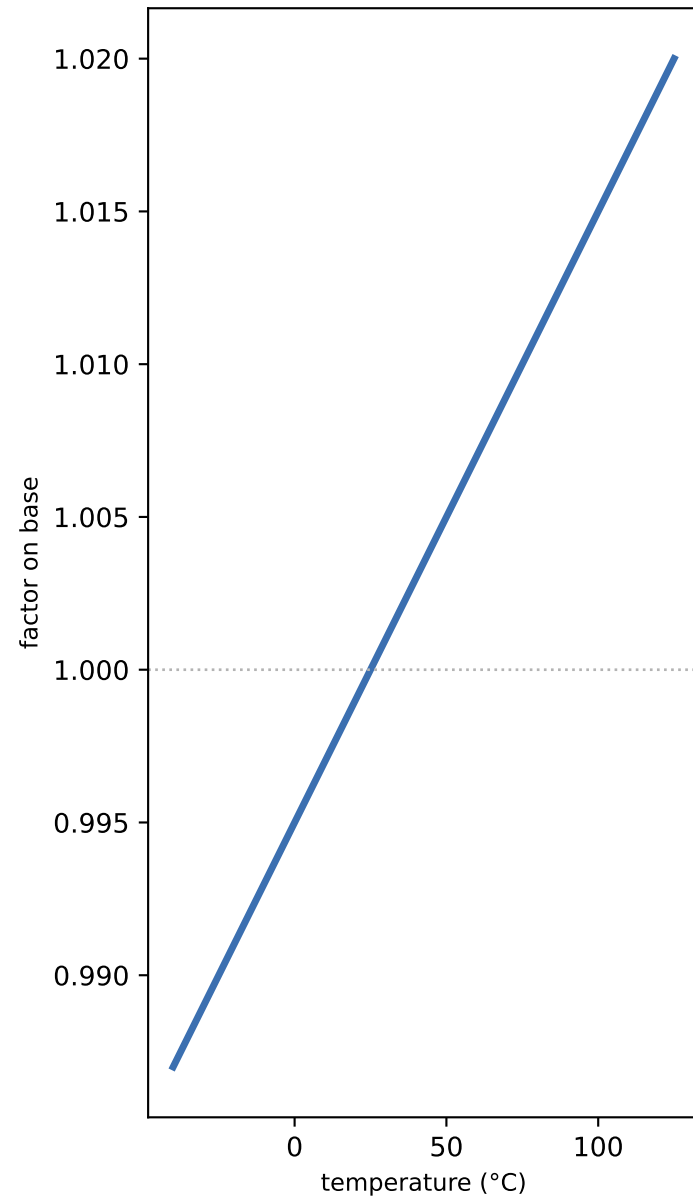
- by_mode** → just a discrete dimension
- by_scenario** → a sampled sensitivity
- insensitive dim.** → constant → automatic broadcast
- the **KeyError** is gone by construction

```
Iload = Quantity(  
    base = normal(100, 8, 'mA'),  
    sensitivities = {  
        'mode': GateToZero(('active',)),  
        'temp': Linear(+0.5%/°C leakage),  
    }) # silent on vbat → constant there  
  
Iload.at(mode='sleep', temp=85) → ~0 mA
```



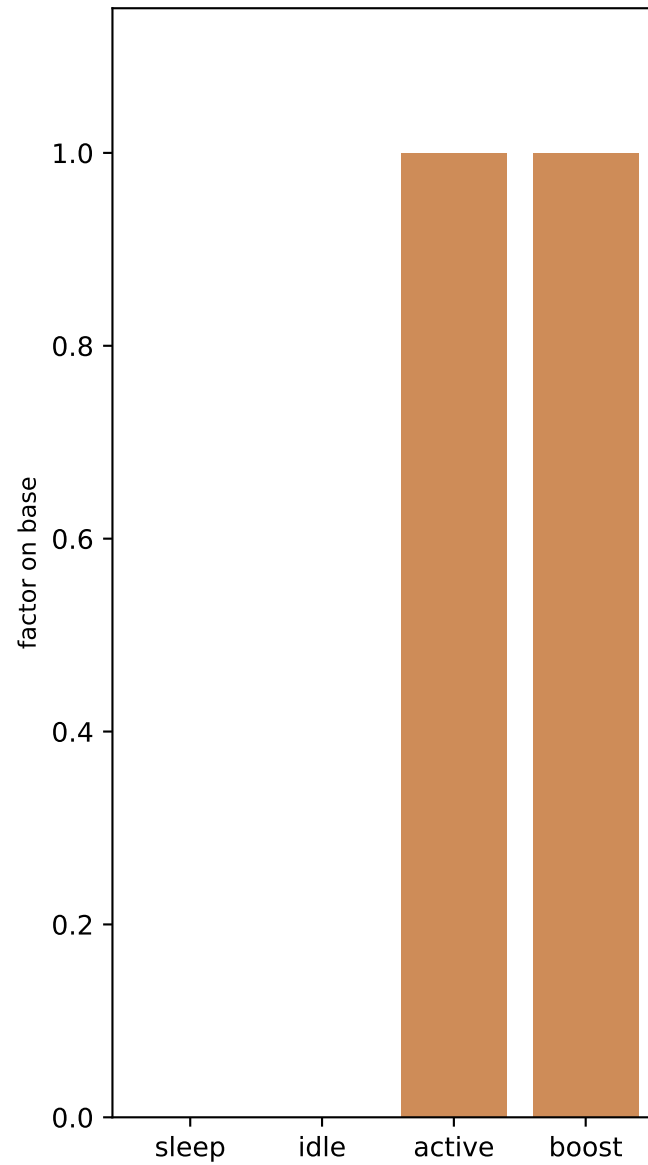
A sensitivity is a function — from a condition to a factor on the base distribution

Linear



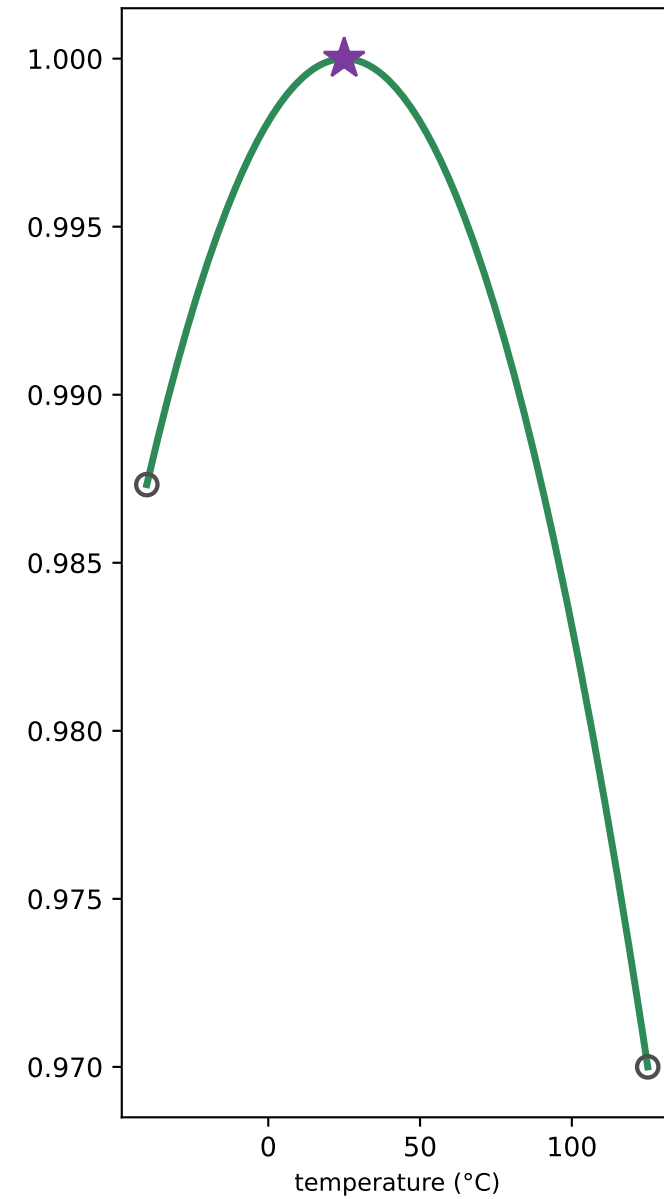
*resistor tempco +200 ppm/°C
monotonic — corners bound it*

GateToZero



*mode gating: 0 unless active
by_mode as a discrete dimension*

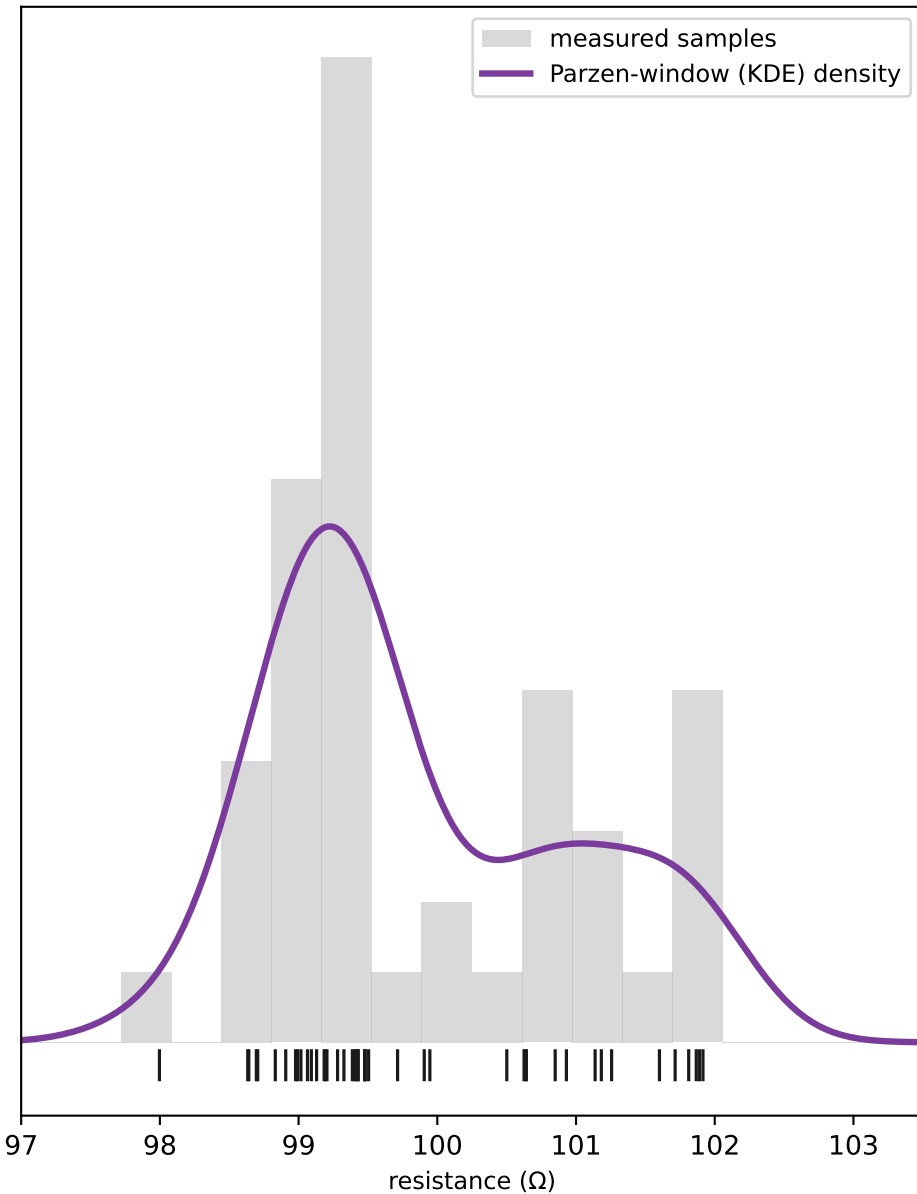
Quadratic (non-monotonic)



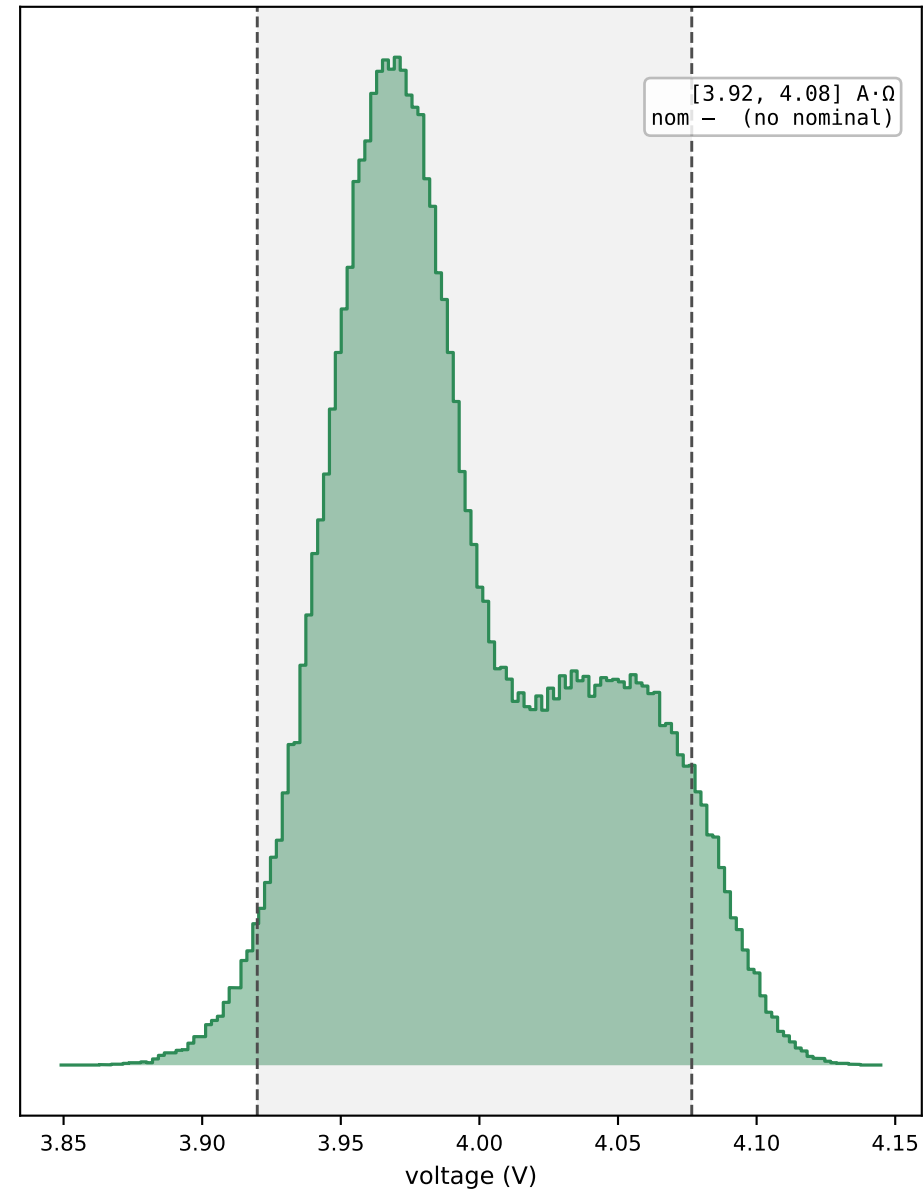
*bandgap curvature: interior peak (★) hides
between corners (○) → flag for MC (\$5)*

Empirical distributions — define a Quantity from measured samples

45 explicit samples → calculated density



used in $V = I \times R$ (nominal undefined)



a characterized part IS a distribution from data; the two-lot structure flows through arithmetic, and (no declared typ) → nominal stays undefined

Characterization vs. contract — same type, two data sources

the contract (synthetic / datasheet spec)
uniform [95, 105] Ω — the promise

characterization (measured, two lots)
where reality actually lands

```
# same type, two constructors  
contract = uniform(95, 105, ' $\Omega$ ')  
char      = from_samples(measured, ' $\Omega$ ')  
#
```

94

96

98

100

102

104

106

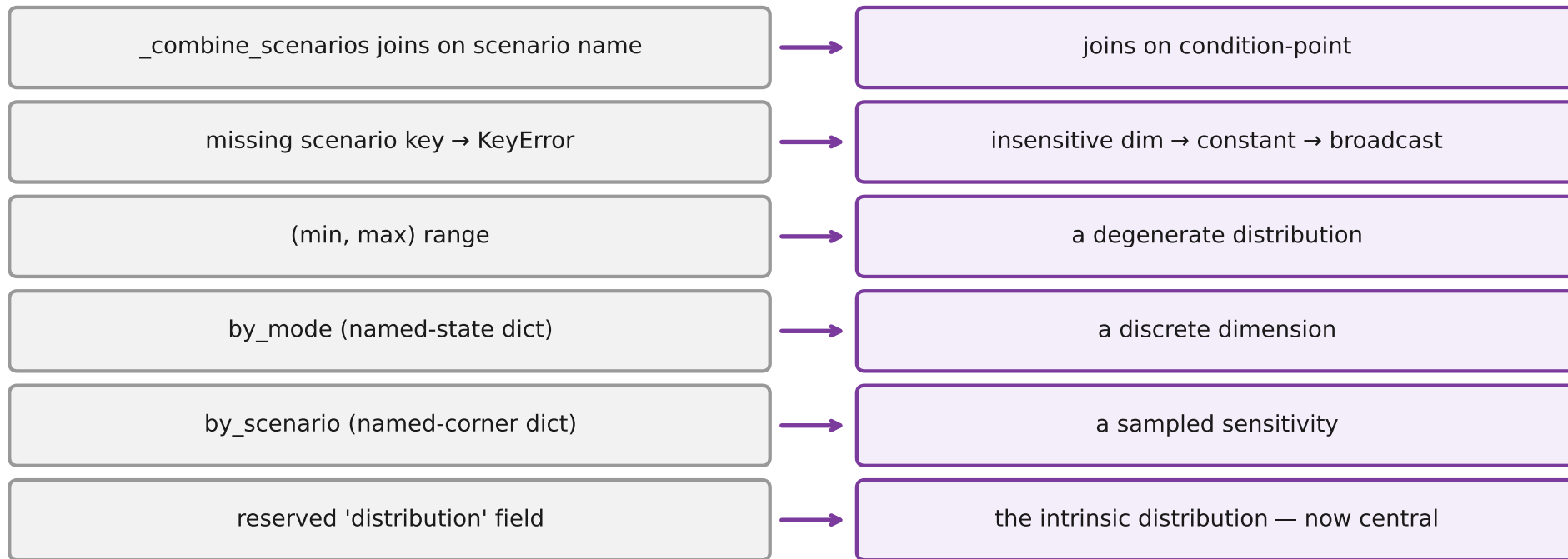
a characterized part IS a distribution + measured sensitivities; the datasheet version is the same shape from a different source

Generalization, not a teardown

the arithmetic core stays; the leaf representation generalizes

v1 (today)

v2



one reconception unifies the characterization = distribution / synthetic spec = contract duality